Creating an Express App: The Solar System

In this tutorial, we'll create a "**Solar System**" website using Express. The website will include a few HTML web pages with images and text.

In this tutorial, we will practice the following:

- Installing Express and related packages
- Creating routes
- Serving static assets such as images, CSS files, etc.
- Creating views
- Installing and using a templating engine
- Passing values from the controller to the views
- Creating and applying partial files ("partials")
- Installing node packages
- Fetching data from a Web API

1. Intro to Express

Express is the most popular Node.js **web application framework**. As such, it's written entirely in JavaScript.
The Express package itself provides just the very basic functionality of a web server, however, there are several additional packages that make it as robust as any other web server (it's able to handle sessions, cookies, interact with databases, etc.)

There are several other **web application frameworks**, and each of them uses a specific programming language. For example, Spring Boot uses Java, Django and Flask use Python, Ruby on Rails uses Ruby, and Laravel uses

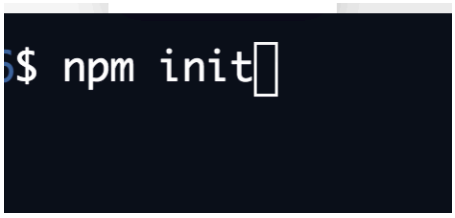
PHP. These frameworks use the MVC (Model-View-Controller) architectural pattern to separate the data model with business rules from the user interface. To learn more visit:
https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Web_frameworks

2. Starting a Basic Express App

Optionally, watch this video tutorial created by a former TA for sections **2 to 10.**

1. Open the terminal window on your computer.
   Create a new folder for your Express app and go inside it:
   $ **mkdir myExpressApp**
   $ **cd myExpressApp**

2. *R*un the command:
   npm init

   This will create the **package.json** file. Use the default values provided by the interactive utility.

   Note: Optionally, you can run this command in "silent" mode, accepting all default values by typing:
   npm init -y

3. Install the Express-related packages, which are **express** and **ejs** (**ejs** is the package for the template engine to create web pages)
   From the terminal window,
   run the command:
   npm i express ejs
   "**i**" is short for "install".

Templating Engines

The Express package is so minimalist that it doesn't include its own functions to render HTML files or web pages. It needs a "Templating Engine".

EJS stands for "Embedded JavaScript Templates". It allows HTML markup to be generated with plain JavaScript.

You can learn more about EJS here: https://ejs.co/

There are many more templating engines such as Pug, Mustache, Handlebars, etc. Each of them has its own syntax.

4. Create a new index.mjs file. Copy and paste the following lines of code into it. This is the basic code of any Express app. Each line is explained below the image.

```
import express from 'express';
const app = express();
app.set("view engine", "ejs");
app.use(express.static("public"));

app.get('/', (req, res) => {
   res.send('Hello Express app!')
});

app.listen(3000, () => {
   console.log('server started');
});
```

The first line imports the Express library.

The second line is based on the Express package documentation, it requires a variable to access the methods. By convention, the variable name is "**app**" but it can be any other name.

The lines in purple represent the "**root route**". This means that when opening the browser in the application's root folder, it should display the message "Hello Express app!". Each "route" maps to a corresponding URL. We'll create more routes later.

In the last three lines, we start the server using port 3000. The second parameter is an anonymous function that displays a message in the terminal indicating that the server has started and that Express is listening for requests.

5. Run the app by typing the following command on the terminal:
   `node index.mjs`

6. Open your browser and go to **localhost:3000**
   You should see the message "`Hello Express app!`" in the browser window.

3. Setting up Templates (Views)

In our current code, the root route uses **res.send()** to display a message in the browser. "**res**" stands for "**response**"; in other words, it's the information the server sends to the browser.

The **send()** method allows for displaying short messages, and it's even possible to include HTML tags. However, it is not practical to display an entire web page using the **send()** method. Instead, we'll use the **render()** method to render an HTML page.

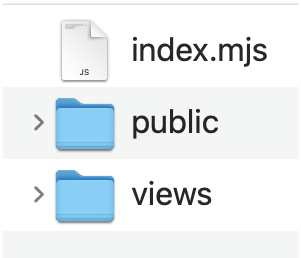
There are a few steps to render a web page:

a) Create the folder structure
b) Set the view engine
c) Specify the folder for static files

a) Create the folder structure

Following the Express framework, all templates (generated HTML files) must be placed within a folder called "**views**". So, create a folder called "**views**" at the same level as the index.mjs file

Also, all static files (such as images, CSS files, etc) should be placed within a folder called, by convention, "**public**". So, create a folder called "**public**" at the same level as the index.mjs file. Make sure that the **public** folder is NOT within the views **folder,** but at the same level.



b) Set the view engine

Given that there are multiple templating engines, our Express app needs to know which one to use. That's why we've added line 3 to the index.mjs file, to indicate that we'll use **EJS** as our templating language:

```
app.set("view engine", "ejs");
```

Keep in mind that you must install the corresponding template engine package first, which we did in the previous section by running:

```
npm i express ejs
```

c) Specify the folder for static files (images, css, etc.)

We're already doing this in line 4 of the index.mjs file:

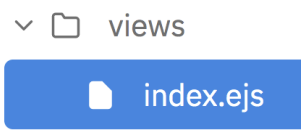
```
app.use(express.static("public"));
```

4. Rendering a Template

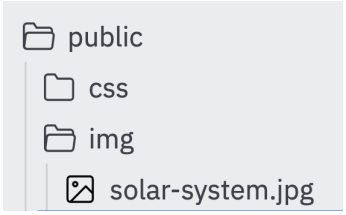
1. In the index.mjs file, change the root route to use the **render**() method instead of **send**(). We'll render a view called "*index*" (but it could have had any name). The **.ejs** extension is optional.

```
app.get('/', (req, res) => {
  res.render('index')
});
```

2. Within the **views** folder, create a new file called "**index.ejs**". The name of the file (index) must match the name passed in the render method.



3. Locate an image of the Solar System (use tools like Pixabay.com or generate one using AI tools). Create an "**img**" folder within the "**public**" folder and put the image inside it.



4. Copy and paste the following HTML code within the **index.ejs** file. Do not include "**public**" as part of the path to the image.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Solar System</title>
</head>
<body>
    <h1>The Solar System</h1>
    <nav><a href="/"> Home </a>
         <a href="/earth"> Earth </a>
    </nav>
    <img src="/img/solar-system.jpg">
</body>
</html>
```

5. Restart the app.
   The app must be run every time the index.mjs file is updated (there is another package to avoid restarting the server, but for now, let's just keep restarting the server). You should see the **index.ejs** file content in the browser window.

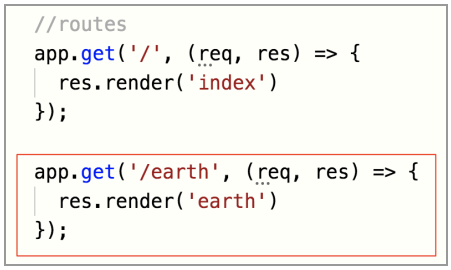
   If you are running the app locally, to stop the server, open the terminal window where the server is running and press **Control+C.**
   Note: For Mac users, it's still Ctrl+C. You might be used to using Cmd, but Ctrl+C is the universal shortcut for terminating console applications.

5. Creating a New Route

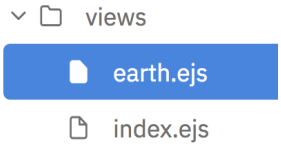
Let us create a second route to display information about Earth.

1. Open the index.mjs file and add the following new route below the root route.

```
app.get('/earth', (req, res) => {
 res.render('earth')
});
```



2. Create a new file within the "**views**" folder.
   The name of the file must be the same as the value of the argument passed to the "render" function (earth) and have the .**ejs** extension.



   Note: The name of the view **does not** need to be the same as the name of the **route**. So, we could have used, for instance, "*planetEarth*" as the name of the view.

3. Copy and paste the following HTML code within the **earth.ejs** file

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Solar System</title>
</head>
<body>
    <h1>The Solar System</h1>
    <nav><a href="/"> Home </a>
         <a href="/earth"> Earth </a>
    </nav>
    <h2>Planet Earth</h2>
</body>
</html>
```

4. Run the application. You should be able to access the Earth page by clicking on the corresponding link and then going back to the “Home” page.

   Notice that the link to the Earth page was created by using the following tag and that the value of “href” corresponds to the name of the route.

   ```
   <a href="/earth"> Earth </a>
   ```

   Likewise, the link to the “Home” page was created using the “root route” as the value of “href”, like this:

   ```
   <a href="/"> Home </a>
   ```

   We could manually add information and an image about planet Earth; however, we’ll use a Node Package instead!

6. Installing a Node Package

There are **four basic steps** to use a Node Package in an Express app:

1. Locate the package. Use search tools such as npmjs.org. Examine the packages found until identifying the appropriate one. In our case, we can use "Solar System" as the keyword. Among the results, we can identify the npm-solarsystem package.

2. Install the package. Run the following command in the Terminal window.

```
npm install npm-solarsystem
```

Optionally, you can use the shortcut version: `npm i npm-solarsystem`
Note: When installing a package, all the package code is saved into the "**node_modules**" folder. You can access and even modify the code!

3. Import the package. There are **two ways** to import node packages, and they can't be used together.

The old way uses the keyword "**require**",
The new way uses the keyword "**import**".

We already used "import" to import the Express package in line 1 of the index.mjs file, so we'll continue using "import". The package's documentation will tell you which method it uses. In the case of the npm-solarsystem package, you'll notice on its documentation that it uses "**require**":

```
const planets = require('npm-solarsystem');
```

So, we need to switch it to use "**import**" by using the following syntax:

```
const planets = (await import('npm-solarsystem')).default;
```

Add the line of code above as the second line of the index.mjs file. It will look like:

```
import express from 'express';
const planets = (await import('npm-solarsystem')).default;
const app = express();
```

Remember: If a package uses "**require**" change it to "**await import**" using the syntax above.

4. Refer to the Package Documentation to use it. The package documentation specifies the names of the methods you can use and their corresponding output. In the case of the npm-solarsystem package, we can use methods such as: .getEarth(), .getVenus(), etc.
The output is an object with properties such as: *image, description, moons.*
Modify the /earth route to get the information for planet Earth:

```
app.get('/earth', (req, res) => {
 let planetEarth = planets.getEarth();
 console.log(planetEarth);
 res.render('earth');
});
```

Restart the server. When going to the /earth route, the terminal window should display the Earth information.

7. Passing Values from Routes to Views

So far, we're displaying the Earth information in the terminal window, however, it should be displayed on the web page instead. We'll need to pass the information from the route to the view.

1. In the last line of the **/earth** route, pass an object with the variable that contains the information:

```
res.render('earth', {"propertyName":planetEarth});
```

propertyName can be any made-up name, such as "earthInfo", "earth", "planetInfo", etc. Optionally, it can be omitted to indicate that the property will have the same name as the variable. Change the line above with the following simpler version:

```
res.render('earth', {planetEarth});
```

2. On the **earth.ejs** file, use the *propertyName* within ejs tags: **<%= %>**
   To display the planet description, use: **<%= planetEarth.description %>**
   To display the planet image, use:
   <img src="**<%= planetEarth.image %>**">

Practice!

Try displaying the information for Mars!
You'll need to create a new **/mars** route and a new **mars.ejs** view.
Don't create routes for other planets yet, since we'll make a few changes before moving on with the rest of the planets.

You'll notice that the Mars image is not displayed. The URL to the Mars picture in the `npm-solarsystem` package is broken. The good news is that when we install a node package, we have access to its code, so let's open the corresponding `npm-solarsystem` file to fix the broken URL.

All packages are installed in the "**node_modules**" folder. So, let's open this folder and locate the "**npm-solarsystem**" folder, and then open the **index.js** file. In line 56, locate the URL to the Mars image. Search for a Mars picture you can use and locate its URL (like this one from Wikipedia). Replace the image value and save the file.

```
54 module.exports.getMars = function Mars() {
55     const mars = {
56         image: 'https://www.nasa.gov/sites/default/files/thumbnails/image/edu_what_is_mars.jpg',
57         distanceFromSun: '145.925 million miles',
58         yearLength: '687 Earth Days',
59         description: 'The fourth planet from the Sun, Mars is a dusty, cold, desert world with a very th
60         radius: 2106,
61         websiteLink: 'https://solarsystem.nasa.gov/planets/mars/overview/',
62         oneEarthDay: '24.6 hours',
63         moons: 2
64     }
65
```

8. Adding Styles

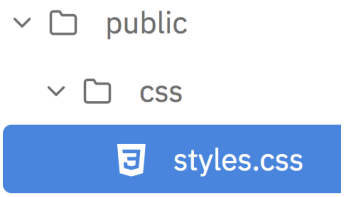
In the Express framework, all static files (images, external CSS files, external JavaScript files, etc.) must be included in a separate folder (they won't be embedded if they're located within the **"views"** folder). By convention, all

static files are included in a folder called "**public**" (but you can use a different folder name).

We have already configured the use of the "public" folder when we added the line: `app.use(express.static("public"));`

1) Let's add a couple of CSS rules in an external CSS file.
   Within the "**public**" folder, create a "**css**" folder.

   Within the "**css**" folder, create a new empty file called "**styles.css**"



2) Type the following CSS rule in the **styles.css** file. You can add any other styles you want or use a different background color (colors can be selected using an [online color picker tool]()

```
body {
    background-color: #CBE2F6;
    text-align: center;
}
```

3) Open the **index.ejs** file and embed the external CSS file within the <head> section using the code below:

```
<head>
    <title> The Solar System </title>
    <link rel="stylesheet" href="/css/styles.css" />
</head>
```

   Observe that the path to the file starts with a forward slash (the root folder) and that "public" is NOT part of the path: **/css/styles.css.**

4) Save the **index.ejs** file and reload it in the browser.
   You don't need to restart the app if the index.mjs file is not modified.

9. Using “Partials”

Partials are reusable, embeddable content that can be used across multiple web pages, such as the navigation or footer.

1) Create a folder called “**partials**” within the “**views**” folder.

2) Create a new empty file called “**head.ejs**” within the “**partials**” folder.

3) Cut the **<head>** element, lines 3 to 7, from the “**index.ejs**” file and paste them into the “**head.ejs**” file.

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <meta charset="utf-8">
5    <title>Solar System</title>
6    <link href="/css/styles.css" rel="stylesheet">
7  </head>
8  <body>
```

4) In the “**index.ejs**” file, include the “**head.ejs**” file by pasting this line of code in line 3:
`<%- include('partials/head.ejs') %>`

5) Do the same for the **<nav>** element.

6) Add a **<footer>** partial as well

EJS Tags
The EJS templating engine uses the tags **<%** and **%>**
To render HTML tags, we must use a hyphen after the open EJS tag: **<%-** `include('partials/head.ejs')` **%>**

Try replacing the hyphen with an equal symbol:
<%= `include('partials/head.ejs')` **%>**
You’ll notice that the HTML tags are displayed rather than applied.

10. Display a Random Background Image

One of the main benefits of using server-side programming is the ability to hide sensitive information, such as Web API Keys. We'll get a random image from a Web API and display it as the background of the **index.ejs** file.

1. Install the node-fetch package.

   In the terminal window, type: `npm install node-fetch`

2. On the index.mjs file, import the node-fetch package right below importing express:

```
import express from 'express';
import fetch from 'node-fetch';
```

3. On the index.mjs file, modify the "/" route by adding the code below. Notice that the anonymous function is preceded by the **async** keyword because it uses **await**.

```
app.get('/', async(req, res) => {
      let apiKey = "7756a1e81f817c186cf57294e1c19b37b49c54b8f34e7c499ee0ce5cd86cd16e";
     let url =
`https://api.unsplash.com/photos/random/?client_id=${ap
iKey}&featured=true&query=solar-system`;
    let response = await fetch(url);
    let data = await response.json();
    let randomImage = data.urls.full;
    res.render("index",{"image":randomImage})
});
```

The code above uses the "Unsplash" Web API, which provides only 50 hits per hour. If you get a "rate exceed" error, consider getting your own API Key by registering as a developer and then creating an app.

Notice that in the last line of the code above, we're passing an object as a parameter to the "index" view. Optionally, we could have simply used:
`res.render("index",{randomImage})`

4. Modify the **index.ejs** file to add the following <style> block right below the "**head.ejs**" partial. Observe that we're using the `image` object property that we passed in the last line of the root route.

```
<%- include('partials/head.ejs') %>
<style>
        body{
            background-image: url("<%= image %>");
            background-size: cover;
        }
</style>
```

5. Restart the app. You should see a random background image when the page loads.

11. Refactoring the **index.mjs** file

So far, we've created two routes and two views to display the information for two planets: Earth and Mars. We could continue creating routes and views for the remaining planets…but we would not be using the full potential of the templating language. Let's refactor our code to retrieve the information for **all** planets with just **three** lines!

1. Start by creating the following **/planet** route:

```
app.get('/planet', (req, res) => {
 let planetName = req.query.planetName;
 let planetInfo = planets[`get${planetName}`]();
 res.render('planet', { planetInfo, planetName });
});
```

The `/planet` route has to be accessed using a query string in the URL, something like: localhost:3000`/planet?planetName=Earth`

The first line of code stores the planet name passed in the query string into the **planetName** variable.

```
let planetName = req.query.planetName;
```

> When sending data through the URL (query string), you can access it in Express using **req.query**. For example, if the URL is **/search?term=**javascript, you can access the value with **req.query.term**

The second line of code uses the bracket notation to invoke a function.
Previously, we had: `planets.getEarth()`
Using the bracket notation: `planets["getEarth"]()`

However, instead of "Earth" we need to use the name of the planet that we're getting through the URL (which is in the **planetName** variable).
That's why the second line uses: `planets[`get${planetName}`]();`

In the third line, we're passing the **planetInfo** object and the **planetName** variable to the "planet" view:
`res.render('planet', { planetInfo, planetName });`

2. Create the `planet` view
   You can duplicate the **earth.ejs** file and rename it to **planet.ejs**
   Replace **planetEarth** with **planetInfo**
   Use **planetName** to display the planet name in the header

3. Modify the **nav.ejs** file
   Replace `<a href="/earth"> Earth </a>`
   With `<a href="/planet?planetName=Earth"> Earth </a>`
   Make sure to capitalize the planet names since the function names are capitalized. Do the same for all planets.

4. Save and test it!

12. Complete the remaining requirements on your own. Refer to the rubric in Canvas.

13. Running an Express app with Nodemon (optional)

By default, the app must be restarted every time the index.mjs file is modified. There are some packages that prevent it, such as nodemon.

To run an Express app with nodemon:

1) Install nodemon globally by typing:
   `npm install -g nodemon`

2) Run your app by typing:
   `nodemon index.mjs`

Now, whenever the index.mjs file is modified, the server will restart automatically.